

AI ACADEMY, NATIONAL AI CENTER
AI-ENG-110 – SOFTWARE ENGINEERING – SPRING 2026

Final Project Report

Topic 4 – Async Research Assistant

Team: Research Assistant Team **Date:** 20 September 2026
Repo: <https://github.com/ruslan-sadigov/research-assistant-SWE-Final-Project> **Tag:** v1.0-final
Members: **Ruslan Sadigov** (@ruslan-sadigov), **Nihat Ismayilzade** (@NihatIsmayilzade),
Samur Eyyubov (@eyyubovsamur240-afk), **Polad Ibrahimli** (@Polad-Ibrahimli)

Executive Summary

We built an asynchronous research assistant: a command-line tool that takes a question, collects evidence from Wikipedia, arXiv and a web-search provider at the same time, and asks an LLM for an answer whose numbered citations point to that evidence. Around the unchanged instructor-supplied `ai/` package we added validated configuration, a concurrency layer with one deadline per source, bounded retries with arXiv request pacing, a SQLite evidence cache, structured logging, 269 tests of our own at 98.58 % coverage, and a multi-stage Docker image of 218.67 MB. In a controlled offline benchmark over the five supplied questions, concurrent collection took 0.31 s against 0.62 s sequentially, the ideal $2.0\times$ for our source delays; against the live services it was faster for four of the five questions ($0.79\text{--}1.82\times$). Our headline robustness case is a live run in which arXiv timed out: the user still received an answer built from Wikipedia’s evidence, with a warning naming arXiv. The most surprising lesson was how much the supplied fetchers depend on the surrounding layer: called directly, Wikipedia rejected them (HTTP 403, no User-Agent) and arXiv answered with a redirect, and full-sentence questions find nothing in Wikipedia search. Starting again, we would rewrite questions into short topic queries and put provider settings into the cache key from day one.

Contents

Executive Summary	1
1 Project Overview	2
1.1 Problem framing & scope	2
1.2 Provider choice and the AI module contract	3
2 Architecture	3
2.1 Module map	3
2.2 OOP design & key abstractions	4
2.3 Data flow across module boundaries	4
3 Concurrency & Performance	5
3.1 Why this concurrency model	5
3.2 Sequential-vs-concurrent benchmark	5
3.3 Rate limit & politeness	6
4 Robustness & Error Handling	6
4.1 Wrapping the AI module	6

4.2	Failure-mode analysis (one concrete incident)	6
4.3	Input validation	7
5	Storage & Caching	7
5.1	Schema	7
5.2	Caching policy	8
6	Testing	8
6.1	Strategy & coverage	8
6.2	Notable tests	9
7	Deployment & Reproducibility	9
7.1	Docker image	9
7.2	Configuration	9
8	Limitations & Future Work	10
9	Tools & Acknowledgements	10
	References	11
A	Appendix — Reproducing the benchmark	11

1. Project Overview

1.1 Problem framing & scope

A user wants a quick, sourced answer to a research question without opening three websites. The system accepts the question and a selection of sources, retrieves evidence from each selected source concurrently, and prints an answer in which every bracketed number refers to a listed reference. A source that fails or times out must not take the others down: the answer is produced from whatever evidence arrived, and the failure is reported as a warning.

Inputs: the question (at most `MAX_QUESTION_LENGTH` = 2000 characters after trimming), `--sources` with any of `wiki`, `arxiv`, `web`, an optional `--no-cache` flag, and settings from environment variables or `.env`. **Outputs:** the answer, numbered references, per-source warnings, and an exit code (0 answer produced, 1 no answer, 2 invalid input or configuration); retrieved evidence is cached in SQLite for 24 hours. **Entry point:** one CLI command, `python -m researcher ask "<question>" --sources wiki,arxiv,web`, also the entry point of the Docker runtime image.

Scope decisions. We deliberately built no HTTP API or UI: a single-user CLI met the topic, and every added entry point would need its own validation and tests. We tightened the specification by validating every input before any network call, escaping control characters in all printed provider text, and treating an empty result as a valid outcome rather than an error. We loosened it in one place: we do not rewrite questions into search keywords, so Wikipedia often returns nothing for full questions (Section 8).

1.2 Provider choice and the AI module contract

Role	Provider	Identifier / setting
LLM (synthesis)	Google Gemini (google-genai SDK)	LLM_PROVIDER=gemini LLM_MODEL=gemini-3.5-flash-lite
Web search	Tavily	WEB_SEARCH_PROVIDER=tavily
Encyclopedia	Wikipedia search API	no key
Papers	arXiv export API	no key
Embeddings	not used	–

Only AIService calls the supplied package: `ai.fetch_wikipedia`, `ai.fetch_arxiv` and `ai.fetch_web` (each with `client=` and `max_results=`), and `ai.synthesize`. Other modules only use its data types `Source`, `Citation` and `AnswerWithCitations`, and `ProviderError` for error classification. *We did not modify the public interface, or any file, of the provided `ai/` package.*

Malformed responses. The supplied synthesizer already drops citation indices that do not refer to a provided source. Our layer adds three checks: arXiv responses whose entries point to arXiv’s `/api/errors` feed are rejected as a provider error, because the supplied parser would otherwise return an error message as a “paper” and we would cache it; every application model rejects unknown fields and enforces invariants (an `ok` outcome must contain sources, a failed or timed-out one must carry a warning and cannot be a cache hit); and cached rows are re-validated on every read. What we cannot detect is a well-formed answer that is wrong: citations show which evidence was used, not that the answer is true.

2. Architecture

2.1 Module map

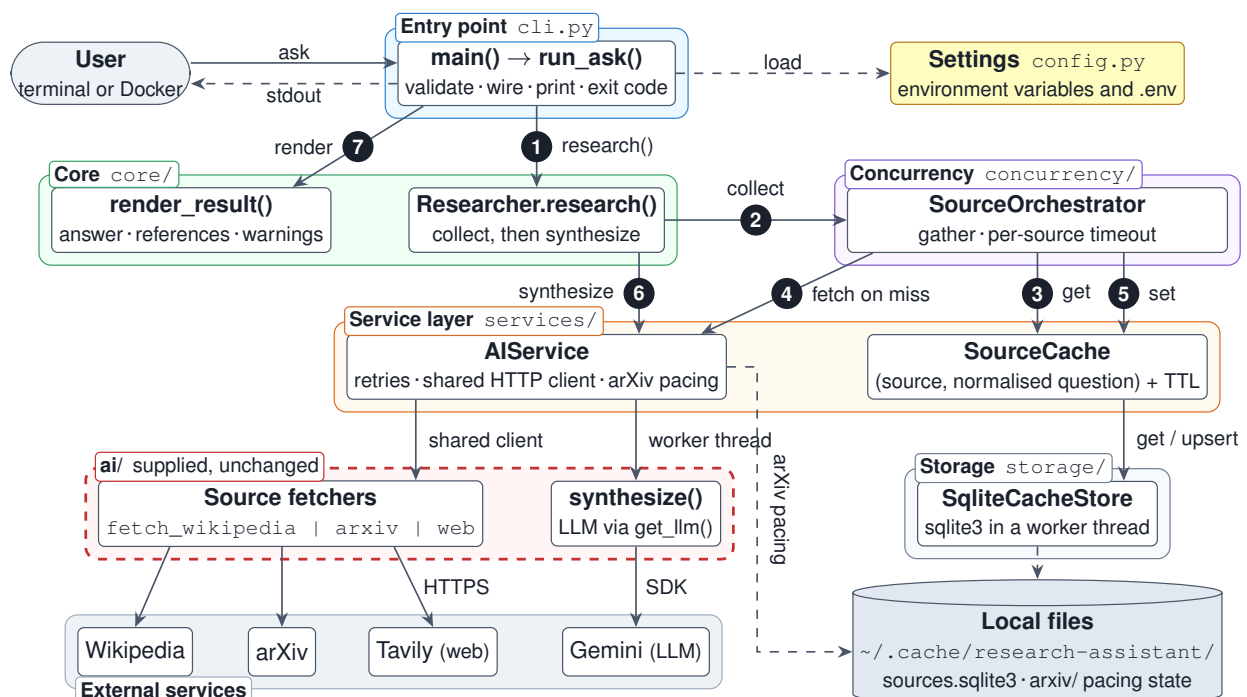


Figure 1: Runtime architecture of one `ask` request. Solid arrows are calls from caller to callee, numbered in request order; dashed arrows are configuration, local files or output. Tab paths are relative to `src/researcher/`. Source: `docs/figures/architecture.tex`.

The **entry point** (`cli.py`) validates input and is the only place that wires concrete classes. The

core (`core/researcher.py`) first collects evidence, then synthesizes, and turns missing evidence or a failed synthesis into warnings. The **concurrency layer** (`concurrency/orchestrator.py`) runs the selected sources together, each under its own deadline, and performs the cache-read, fetch and cache-write sequence. The **service layer** wraps the supplied package (`services/ai_service.py`, with arXiv pacing in `arxiv_limit.py`) and owns cache keys and expiry (`services/cache.py`). The **storage layer** (`storage/cache_store.py`) only reads and writes rows.

Interpreting the diagram. Two arrows cross the `ai/` boundary, both from `AIService`; the shared HTTP client also crosses it in the other direction as the `client=` argument, so the supplied fetchers send their requests through our retrying transport, User-Agent header and redirect handling. Only `SourceCache` crosses into storage, through `CacheStoreProtocol`; `AIService` also writes arXiv pacing files. Swapping Gemini for OpenAI or Anthropic changes *no Python file*: provider selection lives inside `ai/` and reads `LLM_PROVIDER`. It changes `.env` and adds the SDK to `pyproject.toml` and `uv.lock`.

2.2 OOP design & key abstractions

We chose composition with `typing.Protocol` interfaces over abstract base classes. Components receive collaborators through their constructors, and `run_ask()` in `cli.py` wires them:

```

1 # interfaces.py -- a structural contract, no base class
2 class CacheStoreProtocol(Protocol):
3     async def get_entry(
4         self, source: SourceName, query_key: str
5     ) -> CacheEntry | None: ...
6     async def upsert_entry(self, entry: CacheEntry) -> None: ...
7 # cli.py, run_ask() -- the only place concrete classes are wired
8 cache = SourceCache(store, settings.cache_ttl_seconds)
9 orchestrator = SourceOrchestrator(settings, ai_service, cache)
10 researcher = Researcher(orchestrator, ai_service)

```

Why Protocols rather than ABCs. The parts vary independently: two stores (`SQLiteCacheStore` and `InMemoryCacheStore`) sit behind one `SourceCache`, and the tests replace the AI service, cache and store with small fakes that satisfy the protocols structurally, without inheriting from anything or patching modules; mypy still checks the signatures. An ABC would add a base class to every fake for no runtime benefit, and a plain callable would not cover multi-method objects such as a store. Inheritance appears only where a framework expects it: Pydantic models and settings, exceptions (`HTTPRetryExhausted` extends the supplied `ProviderError`, so callers still see a provider error), and `RetryingTransport`, which subclasses `httpx.AsyncBaseTransport` but *wraps* the real transport by composition, so retries apply to any transport, including test mocks.

2.3 Data flow across module boundaries

Our own boundary types are Pydantic models rather than dataclasses, because they validate on construction: `Settings`, `SourceOutcome` (source, status `ok/empty/failed/timeout`, sources, elapsed time, `cache_hit`, warning), `CollectionResult` (outcomes, de-duplicated sources, elapsed time), `ResearchResult` (question, collection, answer, warnings) and `CacheEntry` (source, normalised key, sources, `created_at`, `expires_at`); `SourceName` and `SourceStatus` are `Literal` types. *No naked dictionaries cross module boundaries*; the only JSON is the serialized `CacheEntry` inside the `SQLite` store.

We reused `ai.Source` and `ai.AnswerWithCitations` unchanged, because they already describe evidence and answers. We wrapped them when a layer needed information the supplied types cannot carry: per-source status, timing, cache hits and warnings (`SourceOutcome`), and persistence metadata (`CacheEntry`).

3. Concurrency & Performance

3.1 Why this concurrency model

The work is almost entirely waiting on network I/O, so we use `asyncio`: `SourceOrchestrator` starts one task per unique selected source with `asyncio.gather`, and each task runs inside `asyncio.timeout` (`PER_SOURCE_TIMEOUT_SECONDS`, default 10s), which covers its cache read, fetch attempts, retry waits, arXiv pacing and cache write. The bottleneck we target is the sum of source latencies: sequentially a question waits for every source in turn, concurrently only for the slowest. The **bound** is structural: at most one task per source, so three per question; no semaphore is needed. Each task catches its own failures and returns an outcome, which is why we use `gather` rather than `TaskGroup` (which cancels siblings on the first error). Blocking work leaves the event loop through `asyncio.to_thread`: SQLite operations and `ai.synthesize`.

Threads would also work for three I/O calls, but `asyncio` gives us one shared HTTP client, cancellable deadlines and deterministic tests without locks around shared state. Concurrency pays off as soon as two sources are selected, since their waits overlap. A semaphore of 1 would make the design sequential; a semaphore of 100 would change nothing, because there are never more than three tasks.

3.2 Sequential-vs-concurrent benchmark

Offline (`benchmarks/benchmark_sources.py`): the real orchestrator with simulated source delays of 0.1, 0.2 and 0.3s, one warm-up excluded, median of five alternating runs; recorded 2026-09-16 on Windows 11 (AMD64), Python 3.12.11. **Live** (`benchmarks/benchmark_live_sources.py`): real Wikipedia, arXiv and Tavily requests, one measured pair per question, 3.1s pause before each collection; recorded 2026-09-16 on Windows, Python 3.12.11. Both use the five supplied questions with all three sources ($N = 3$ requests per question, 15 per mode) and *no cache*; synthesis is excluded. Commands: `uv run python benchmarks/benchmark_sources.py --repeats 5` (offline) and `uv run python benchmarks/benchmark_live_sources.py --repeats 1` (live); the full reproducer is in the appendix.

Mode	Workload	N	Sequential (s)	Concurrent (s)	Speedup
Offline	q1 Photosynthesis	3	0.6282	0.3144	2.00×
Offline	q2 Long context windows	3	0.6238	0.3126	2.00×
Offline	q3 Financial crisis	3	0.6227	0.3135	1.99×
Offline	q4 Fusion energy	3	0.6239	0.3133	1.99×
Offline	q5 CRISPR-Cas9	3	0.6226	0.3117	2.00×
Live	q1 Photosynthesis	3	6.009	3.616	1.66×
Live	q2 Long context windows	3	4.411	5.596	0.79×
Live	q3 Financial crisis	3	6.521	3.624	1.80×
Live	q4 Fusion energy	3	4.550	4.343	1.05×
Live	q5 CRISPR-Cas9	3	7.044	3.868	1.82×

Interpretation. $N\times$ is not the right ceiling here: with unequal delays the ideal is $sum/maximum = 0.6/0.3 = 2\times$, which the offline run reaches within scheduling overhead. Live, every run returned the same outcome (Wikipedia empty, three arXiv and three Tavily results, six unique URLs, identical in both modes, no failures or timeouts). Concurrency won four of five pairs; q2 was slower, and a single pair cannot tell whether network variance, provider caching or arXiv’s pacing caused it. The bottleneck is network latency and arXiv’s 3s request spacing, not the GIL or the database (the cache was off). Going further would mean running several questions at once or rephrasing queries, not adding workers.

3.3 Rate limit & politeness

Two mechanisms apply. (1) **Sleep on retryable errors:** HTTP 408, 429 and 5xx, transport errors and timeouts are retried up to `RETRY_MAX_ATTEMPTS = 3` attempts in total, waiting 0.5s, then 1s, doubling up to `RETRY_MAX_DELAY_SECONDS = 4s`, or longer when the server sends `Retry-After`. (2) **arXiv pacing:** following arXiv’s terms of use [4], a file lock under `~/.cache/research-assistant/arxiv/` serialises arXiv requests across processes and leaves at least 3s after the previous request completed; a `Retry-After` from arXiv is stored as a cooldown for all processes. Wikipedia requests carry a descriptive User-Agent, as its policy requires [5].

The offline test suite makes no real provider calls; its largest burst is three simulated source calls per question. We have not observed an HTTP 429 from a live provider; the path is exercised with mocked 408, 429, 500 and 503 responses. We did observe intermittent Gemini 503 (“high demand”) responses, which take the same retry path and are logged as, for example, `AI operation failed: operation=synthesis attempt=1 max_attempts=3 error_type=... http_status=503`.

4. Robustness & Error Handling

4.1 Wrapping the AI module

Retries. Retryable: HTTP 408/429/5xx, `httpx` transport errors, timeouts, connection errors and SDK connection or timeout errors. Not retried: other 4xx, invalid values, missing packages, too many redirects, and supplied `ProviderErrors` that signal setup problems (a key “is not set”, an unknown provider, malformed XML). HTTP requests are retried inside the transport; when that budget is exhausted it raises `HTTPRetryExhausted`, which the outer fetch retry does not repeat, so one source never makes 3×3 requests. **Timeouts.** The HTTP client uses the per-source value, and the orchestrator’s `asyncio.timeout` bounds the whole source. Synthesis has no separate deadline: its attempts are bounded by the retry settings, and an SDK call already running in a worker thread cannot be cancelled. **Logging.** Every failed attempt logs `operation`, `attempt`, `max_attempts`, `error_type` and `http_status` at WARNING; each source logs `source`, `status`, `cache_hit`, `results` and `elapsed_seconds` at INFO; timeouts log `source`, `stage` and `deadline_expired`. Our own records never contain exception messages, headers or keys; library loggers such as `httpx` still record request URLs at INFO. **Validation** is described in Section 4.3.

Walk-through: HTTP 429 from Tavily. The transport sees 429 and raises; the retry waits the larger of 0.5s and `Retry-After`; a 200 on the second attempt returns results as if nothing happened. If all three attempts fail, the web source ends as `failed` with the warning “web could not retrieve evidence.”, while Wikipedia and arXiv evidence is still used.

4.2 Failure-mode analysis (one concrete incident)

(i) **Injection.** We pointed `DATABASE_URL` at a file containing plain text instead of a SQLite database and ran the real CLI for `q1` with all three sources (`tests/cache_verification.py`; providers simulated, network blocked). (ii) **System behaviour.** For each source, the cache read raised `CacheStoreError`; the orchestrator logged it, fetched fresh evidence, tried to save it, failed again, and kept the evidence. Synthesis ran and the exit code was 0. The file was left byte-for-byte unchanged. (iii) **What the user saw:** the normal answer and references, followed by

Warnings:

- wiki cache could not be read; fetching fresh evidence. wiki results could not be saved to cache.
- arxiv cache could not be read; fetching fresh evidence. arxiv results could not be saved to cache.
- web cache could not be read; fetching fresh evidence. web results could not be saved to cache.

(iv) **Logs** (timestamps removed):

```

WARNING researcher.concurrency.orchestrator: cache read failed: source=wiki; attempting fresh fetch
WARNING researcher.concurrency.orchestrator: cache write failed: source=wiki; fetched evidence
retained
INFO researcher.concurrency.orchestrator: source completed: source=wiki status=ok cache_hit=False
results=1 elapsed_seconds=0.002

```

A natural assumption is that a broken file is detected once per run; in fact every read and write retries opening the file, so each run produces two warnings per source until the file is removed. The logs name the source and stage but, deliberately, not the path, so diagnosing requires checking DATABASE_URL. We documented the missing repair as a finding rather than changing the working cache code. The same degradation happened in a **live** run where arXiv hit its deadline: the answer was built from Wikipedia’s evidence and printed with an arXiv warning (reproduced offline by `test_deadline_cancels_slow_fetch_and_preserves_success`).

4.3 Input validation

Entry	Rule	User message (exit code 2)
Question	Trimmed; not empty; at most MAX_QUESTION_LENGTH characters	Invalid input: Question must not be empty. / ... Question is N characters, which exceeds the maximum of 2000.
--sources	Comma list, case-insensitive, known names only, duplicates removed, at least one	Invalid input: Unknown source(s): x. Valid: wiki, arxiv, web / Select at least one source.
Environment	Typed and range-checked (positive, finite, max delay $\geq$ initial delay, known log level); values never echoed	Invalid configuration; check environment settings and the .env file.
DATABASE_URL	sqlite:/// plus a file path; no :memory:, ?, # or NUL	Invalid cache configuration; DATABASE_URL must be a SQLite file URL.
Output	Control and formatting characters in provider text are escaped before printing	–

A 50,000-character question is rejected before any network call. There is no file or image input. DATABASE_URL may point anywhere the user can write: it is operator configuration on the user’s own machine, not remote input.

5. Storage & Caching

5.1 Schema

Read back from a database the application created (PRAGMA user_version = 1, rollback-journal mode):

```

CREATE TABLE cache_entries (
  source TEXT NOT NULL,      -- 'wiki' | 'arxiv' | 'web'
  query_key TEXT NOT NULL,   -- question after normalize_query()
  payload TEXT NOT NULL,     -- whole CacheEntry as JSON
  PRIMARY KEY (source, query_key)
);
-- index sqlite_autoindex_cache_entries_1: unique, created for the primary
  key

```

The composite primary key’s automatic unique index is the only index; it serves the only query the store runs (WHERE source = ? AND query_key = ?) and turns a repeated write into an in-place update. There are **no foreign keys**, because there is one table. **Every column is derived:**

rows are copies of what the external services returned at fetch time, and those services remain the source of truth, so deleting the file costs speed and API quota, not information. `query_key` is derived from the question, and `created_at/expires_at` inside the payload are cache metadata. We store the entry as JSON rather than normalised columns because it is always read and written whole and validated by `CacheEntry` on read; the price is that expiry is invisible to SQL. `tests/test_cache_store_contract.py` fails if this layout changes.

5.2 Caching policy

What: source evidence per (source, normalised question); answers are never cached, so every run synthesizes. **Key normalisation:** Unicode NFKC, collapsed whitespace, trailing `?!.` removed, case-folded. **How long:** `CACHE_TTL_SECONDS` = 86 400 (24 h); an entry is a miss from its expiry second. Empty results are cached too; failures and timeouts are not. **Where:** `~/.cache/research-assistant/sources.sqlite3` by default, or `DATABASE_URL`; in Docker, a named volume at `/home/appuser/.cache/research-assistant`. `--no-cache` never opens the file. **Invalidation:** expiry only; an expired row is replaced by the next successful fetch of the same key; otherwise delete the file or change `DATABASE_URL`.

Hit rate (hits / eligible lookups), measured by `tests/cache_verification.py` through the real CLI with simulated providers, five questions $\times$ three sources: **0/15 on the first run, 15/15 on the second, 15/30 = 50 % combined**; all ten runs still synthesized an answer. Reformatted questions (other case, spacing and trailing punctuation) also hit 15/15, and a `--no-cache` pass made no lookups and left the file unchanged. Live, a repeated arXiv query reported `cache_hit=True`, and CI shows a cache hit across two separate containers sharing a volume.

Staleness. Three kinds of entry can mislead: an empty Wikipedia result stays empty for 24 h, entries fetched under another `WEB_SEARCH_PROVIDER` or `MAX_SOURCES_PER_QUERY` are served because the key ignores settings, and sources can change within the TTL. In production we would add a settings fingerprint to the key, a shorter TTL for empty results, an indexed `expires_at` column with a purge command, and a size bound.

6. Testing

6.1 Strategy & coverage

Suite	Result
<code>src/researcher</code> (our SE layer), 269 tests	98.58 % line coverage
Provided <code>ai/</code> smoke tests (16)	passing
Ruff, Black, isort, mypy (15 source files)	clean

All 285 tests run offline (`uv run python -m pytest --cov=researcher`). We mock at two levels: fakes that satisfy `AIServiceProtocol` or the cache protocols, and replacements for the `ai.fetch_*` and `ai.synthesize` functions so the real CLI, service, orchestrator and SQLite store still run. HTTP behaviour is tested with `httpx.MockTransport`; network sends are blocked elsewhere, so the suite makes no real provider calls. CI runs lint, type checks and tests, builds the Docker test image, runs it with `--network none`, and checks the cache across two runtime containers.

Not covered, on purpose: the `__main__.py` shim and the `if __name__ == "__main__"` line, the file-locking branch for the *other* operating system in `arxiv_limit.py` (each platform runs only its own), two rarely reached branches in `ai_service.py` (an SDK connection-error check and a final assertion) and one validator branch in `config.py`. A weakness we accept: some store tests still inspect private attributes (the SQLite connection), which ties them to implementation details.

6.2 Notable tests

1. **Happy path, end to end:** `tests/test_sample_questions.py` runs the real CLI three times for each supplied question (10 cases: all sources and each question's expected subset): first retrieval, SQLite reuse, then `--no-cache`, checking exit codes, answers, references and fetch counts.
2. **Concurrency:** `test_source_failure_preserves_other_evidence` makes `arXiv` raise `RuntimeError` or `TimeoutError` inside `gather`; the outcomes are `ok`, `failed`, `ok`, `Wikipedia` and web evidence is kept in order, and the shared client is closed. A companion test proves the fetches overlap and share one client.
3. **Error path with a real bug:** `test_arxiv_error_feed_rejected_without_retry` feeds an `arXiv` Atom error feed. The supplied parser returns it as an ordinary "paper", which we would have cached and cited; the service now rejects it without retrying. The HTTP retry test (`test_http_retries_and_closes_responses`, statuses 408, 429, 500, 503) also checks that every response is closed.

What we did not test: scheduled live contract tests against the real providers, and many CLI processes sharing one `arXiv` lock and one SQLite file under load.

7. Deployment & Reproducibility

7.1 Docker image

A **multi-stage** build on `python:3.12-slim` (Python 3.12.14 at verification): a **builder** stage installs the locked dependencies with `uv`, a **test** stage adds development tools and tests, and the **runtime** stage copies only the installed environment and runs as non-root `appuser` (UID 1000). The runtime image is **218.67 MB** (218 667 271 bytes), below the 250 MB bonus limit; the test image is 569.65 MB. `uv`, `pytest`, demo files and `.env` are absent from the runtime image.

```
docker build --target test -t research-assistant:test .
docker build --target runtime -t research-assistant:runtime .
docker run --rm --network none research-assistant:test
docker run --rm --env-file .env research-assistant:runtime `
  ask "Photovoltaic effect" --sources wiki,arxiv,web --no-cache
docker volume create research-assistant-cache
docker run --rm --env-file .env --env DATABASE_URL= `
  --mount type=volume,source=research-assistant-cache,target=/home/appuser/.cache/research-
  assistant `
research-assistant:runtime ask "Photovoltaic effect" --sources wiki
```

The base is Debian *slim* (glibc) rather than Alpine (musl), so the prebuilt wheels resolved in `uv.lock` install without compilation. Compared with the bare slim base (~120 MB), the installed environment adds roughly 100 MB. In a live container run, the three sources took 2.39, 3.53 and 0.53s, and collection finished in 3.64s.

7.2 Configuration

Existing environment variables override `.env`; application settings with invalid values stop the CLI with exit code 2.

Variable (default)	Effect when missing
LLM_PROVIDER (gemini in .env.example)	The supplied factory falls back to anthropic ; without its key or SDK, synthesis fails and evidence is printed (exit 1).
LLM_MODEL	The provider's built-in default model.
GOOGLE_API_KEY (or GEMINI_API_KEY, LLM_API_KEY)	Synthesis fails without retrying (“is not set”); evidence is still printed, exit code 1.
WEB_SEARCH_PROVIDER (tavily)	Tavily is used.
TAVILY_API_KEY	The web source fails at once with a warning; other sources continue.
DATABASE_URL (blank)	~/.cache/research-assistant/sources.sqlite3.

The remaining variables all fall back to the default in brackets when missing: LOG_LEVEL (INFO), MAX_QUESTION_LENGTH (2000), PER_SOURCE_TIMEOUT_SECONDS (10), MAX_SOURCES_PER_QUERY (3), RETRY_MAX_ATTEMPTS (3), RETRY_INITIAL_DELAY_SECONDS (0.5), RETRY_MAX_DELAY_SECONDS (4) and CACHE_TTL_SECONDS (86400).

8. Limitations & Future Work

- **Wikipedia and full questions.** Live runs of the five supplied questions returned no Wikipedia matches; short topic queries such as “Photovoltaic effect” worked. With another week we would extract a topic query first.
- **Single LLM, no fallback.** A Gemini outage or quota limit means evidence without an answer. Productionising would need a second provider and quota monitoring.
- **Cache freshness.** Our weakest design decision: the key ignores provider settings, empty results live for the full TTL, and nothing evicts old rows (Section 5).
- **Untested under load.** Many processes sharing arXiv pacing will queue behind the 3s spacing and can hit their 10s deadlines; SQLite serialises writers. On a busy day, provider rate limits and this queue are the most likely breakage.
- **Thin live evidence.** The live speedup rests on one pair per question; we would repeat it at least three times and compare distributions.

9. Tools & Acknowledgements

AI assistance. The table repeats the signed contribution statement, plus one row for this revision. We can defend every line of code; “the AI wrote it” is not an answer we use.

Module / file	Assistant	What we did with it
docs/figures/architecture.*, architecture and cache documents, tests/cache_verification.py, tests/test_cache_*.py; first drafts of this report and the deck	Claude (Anthropic)	Drafted the figure, harness, cache tests and documents. Samur Eyyubov reviewed each file, ran the suite and harness, and corrected facts against the code.
Researcher application code, tests, CI/Docker setup, documentation	OpenAI Codex	Implementation drafts, debugging, test design, CI/Docker configuration and documentation. The responsible members reviewed and adapted the output and verified the behaviour before committing.
report/report.tex, slides/slides.tex (final revision)	Claude (Claude Code)	Aligned both documents with the templates: refreshed figures, added this table, rebuilt the slides. Reviewed in a pull request.

Acknowledgements. The ai/ package, sample questions, demo and smoke tests were supplied by the course. We build on httpx, Pydantic, pydantic-settings, python-dotenv, pytest (with pytest-asyncio and pytest-cov), Ruff, Black, isort, mypy, uv and Docker.

References

- [1] Python documentation, “Coroutines and Tasks” (gather, timeout, to_thread), <https://docs.python.org/3/library/asyncio-task.html>
- [2] HTTPX documentation, transports and clients, <https://www.python-httpx.org/>
- [3] Pydantic and pydantic-settings documentation, <https://docs.pydantic.dev/>
- [4] arXiv API Terms of Use (one request every three seconds), <https://info.arxiv.org/help/api/tou.html>
- [5] Wikimedia User-Agent policy, https://foundation.wikimedia.org/wiki/Policy:User-Agent_policy
- [6] RFC 9110, HTTP Semantics, Retry-After, <https://www.rfc-editor.org/rfc/rfc9110>
- [7] SQLite documentation, PRAGMA statements, <https://www.sqlite.org/pragma.html>
- [8] Tavily API documentation, <https://docs.tavily.com/>
- [9] Google Gemini API documentation, <https://ai.google.dev/gemini-api/docs>
- [10] Docker documentation, multi-stage builds, <https://docs.docker.com/build/building/multi-stage/>

A. Appendix — Reproducing the benchmark

From a clone of the repository with uv installed (PowerShell). The offline commands need no keys; the live benchmark needs TAVILY_API_KEY in .env and consumes web-search quota.

```
git clone https://github.com/ruslan-sadigov/research-assistant-SWE-Final-Project.git
cd research-assistant-SWE-Final-Project
uv sync --locked
uv run python benchmarks/benchmark_sources.py --repeats 5      # offline table
Copy-Item .env.example .env                                   # then add keys
uv run python benchmarks/benchmark_live_sources.py --repeats 1 # live table
uv run python tests/cache_verification.py                       # cache hit rate
uv run python -m pytest --cov=researcher --cov-report=term-missing
uv run python -m researcher ask "Photovoltaic effect" --sources wiki,arxiv,web
```

The same checks run in Docker with the commands in Section 7. Methods and raw data: docs/concurrency-benchmark.md, docs/live-benchmark.md, docs/cache-verification.md and docs/docker.md.

End of report — thank you for reading.